educative

Problem Challenge 2

We'll cover the following

String Anagrams (hard)

Given a string and a pattern, find **all anagrams of the pattern in the given string**.

Every **anagram** is a **permutation** of a string. As we know, when we are not allowed to repeat characters while finding permutations of a string, we get $N!$ permutations (or anagrams) of a string having N characters. For example, here are the six anagrams of the string “abc”:

1. abc
2. acb
3. bac
4. bca
5. cab
6. cba

Write a function to return a list of starting indices of the anagrams of the pattern in the given string.

Example 1:

```
Input: String="ppqp", Pattern="pq"
Output: [1, 2]
```

```
Explanation: The two anagrams of the pattern in the given string are "pq" and "q
p".
```

Example 2:

```
Input: String="abbcabc", Pattern="abc"
Output: [2, 3, 4]
Explanation: The three anagrams of the pattern in the given string are "bca", "ca
b", and "abc".
```

Try it yourself

Try solving this question here:

Java | Python3 | JS | C++

```
import java.util.*;

class StringAnagrams {
  public static List<Integer> findStringAnagrams(String input, String val
    List<Integer> resultIndices = new ArrayList<Integer>();
    List<Character> arr = new ArrayList<>(val.length());
        int c;
        int no=0;
        for(int end =0 ; end< input.length();end++){
            char ch = input.charAt(end);
            arr.add(ch);
            if(arr.size() == val.length()){
                c=0;
                for(char ch1 : val.toCharArray()){
                    if(!arr.contains(ch1))
                        break;
                    c++;
                }
                if(c == val.length()){
                    System.out.println(arr);
                    resultIndices.add(end-val.length()+1);
                    no++;
                }
                arr.remove(0);
            }
        }
```

Test Save Reset

Back Completed Next

Solution Review: Problem Challenge 1

Solution Review: Problem Challenge 2